

MFNNS: A Multiplier-Free Near-Memory Accelerator for Graph-Based ANNS

Rui Meng, Yonghao Chen, Jiaxiang Zou, Jingyu Guo, Chenyu Zhang, Xinyu Chen[†]

The Hong Kong University of Science and Technology (Guangzhou)

Guangzhou, China

{rmeng603, ychen433, jzou521, jyguo012, czhang539}@connect.hkust-gz.edu.cn,

xinyuchen@hkust-gz.edu.cn

Abstract—Approximate nearest neighbor search (ANNS) is a key primitive in vector databases, but remains inefficient on CPUs due to high memory traffic and wasted computation on non-nearest candidates. Recent near-memory processing (NMP) accelerators reduce data movement by moving distance evaluation near memory and employ early termination (ET) to prune non-promising candidates. However, they still rely on multiplication-heavy datapaths in both distance evaluation and ET, shifting the bottleneck from memory access to rank-side computation.

We present MFNNS, a multiplier-free NMP accelerator for graph-based ANNS. MFNNS is built on two key ideas. First, it leverages floating-point multiplication approximation (FPMA) to replace multiplication in distance evaluation, and further specializes the design for the square-dominated structure of ℓ_2 distance, reducing compensation from a generic 2D table to a compact 1D lookup. Second, it exploits exponent-domain information as a lightweight signal for pruning, and redesigns early termination (ET) as order-preserving screening using sign/exponent metadata. This allows many non-promising candidates to be filtered before full-distance evaluation. MFNNS further co-designs the NMP architecture, data layout, and access policy to expose screening metadata early and reduce unnecessary payload accesses. At 90% recall, MFNNS improves energy efficiency by $5.23\times$ over the CPU baseline and $26.06\times$ over the GPU baseline. Compared with the state-of-the-art NMP accelerator, MFNNS delivers $1.8\times$ higher throughput and $3.18\times$ better area efficiency. MFNNS is open-sourced at <https://github.com/CLab-HKUST-GZ/micro59-mfnns>.

I. INTRODUCTION

Approximate nearest neighbor search (ANNS) underpins modern vector databases and powers applications such as information retrieval [1], recommendation systems [2]–[4], and retrieval augmented generation (RAG) [5]–[7]. Although graph-based ANNS (GANNS) [8], [9] prunes the search space through selective graph traversal, search remains dominated by distance computation because each visited candidate still requires fetching high-dimensional vectors from DRAM and comparing them against the query. As a result, even optimized CPU implementations remain fundamentally limited by memory bandwidth rather than compute throughput, because distance evaluation exposes very low arithmetic intensity.

Near-memory processing (NMP) [10]–[15] is a promising approach to addressing the memory bottleneck in GANNS, as it exploits the internal bandwidth of DRAM. Recent NMP-based GANNS accelerators accordingly push distance

evaluation into the DIMM buffer chip [16], [17]. However, once off-chip data movement is reduced, throughput becomes increasingly constrained by the arithmetic resources replicated near memory, making memory-side compute efficiency a first-order design concern.

Early termination (ET) [18]–[20] is a natural complement to NMP because it can reject non-promising candidates before their full vectors are fetched. Existing NMP designs accordingly incorporate ET to improve throughput [16], but they remain multiplier-intensive in both paths. First, distance evaluation still relies heavily on floating-point (FP) multipliers replicated across the buffer chip. Second, ET itself remains too costly, because it is invoked for every visited candidate across multiple rounds of data fetching, its cumulative compute overhead can be substantial, especially in existing designs that prune candidates through repeated partial-distance updates dominated by FP multiplications over fetched vector prefixes [16]. Taken together, existing NMP-based accelerators alleviate the bandwidth bottleneck of CPU-based GANNS largely by replacing it with a compute and area bottleneck centered on FP multiplication.

Our key insight is that GANNS is inherently approximate and primarily requires sufficient ordering fidelity to preserve retrieval quality. This tolerance reduces the need for exact FP arithmetic and opens two complementary opportunities. For distance evaluation, approximate multiplication becomes viable as long as candidate ordering remains stable. Floating-point multiplication approximation (FPMA) has shown promising results in neural networks [21], [22], motivating the investigation of whether similar approximation can preserve ranking quality in GANNS with much lighter hardware. For ET, the goal is not accurate partial-distance accumulation but cheap *screening* that separates clearly non-promising candidates from competitive ones early. This suggests that ET can be driven by sign and exponent information, rather than by repeated multiplication-heavy partial updates. These observations motivate a GANNS-specific NMP design that rethinks approximate distance evaluation and low-cost ET.

- We develop a multiplier-free approximate distance computation mechanism for GANNS that removes FP multiplication from the distance evaluation path, enabling lightweight distance computation while preserving ranking quality.

[†]Corresponding author.

- We propose an exponent-only early termination mechanism for GANNS that prioritizes order preservation over absolute bound tightness, enabling most non-promising candidates to be pruned without mantissa fetches or FP partial updates.
- We co-design the NMP architecture, data layout, and access policy through a metadata-payload split organization and an ET-aware open-row policy, translating cheap screening decisions into reduced DRAM traffic and row activations.
- Comprehensive experiments show that MFNNS delivers average speedups of $29.9\times$ over the CPU baseline and $1.81\times$ over the GPU baseline, while improving energy efficiency by $5.23\times$ and $26.06\times$, respectively. MFNNS also outperforms state-of-the-art NMP accelerators, achieving $1.8\times$ higher throughput, $3.18\times$ better area efficiency, and $1.74\times$ higher energy efficiency.

II. BACKGROUND AND MOTIVATION

A. Graph-based ANNS (GANNS)

Given a vector database $V = \{v_1, v_2, \dots, v_N\}$ and a query vector q , where each vector has D dimensions, nearest neighbor search (NNS) retrieves the top- k vectors closest to q under a given distance metric. A widely used choice is squared ℓ_2 -distance:

$$d^{(E)} = \sum_{i=1}^D (q_i - v_i)^2 \quad (1)$$

Other similarity measures, including inner product and cosine similarity, can be reduced to equivalent ℓ_2 -distance formulations [23]–[25]. A brute-force NNS implementation evaluates the distance from q to all N vectors, resulting in $O(ND)$ computation and comparable memory traffic. This exhaustive search quickly becomes impractical for billion-scale, latency-sensitive retrieval workloads [26], [27].

Approximate nearest neighbor search (ANNS) reduces query latency by trading a small loss in accuracy for substantially less search work [28]–[30]. Among existing approaches, graph-based ANNS (GANNS) is widely adopted in modern systems [8], [9], [31]–[33]. As shown in Figure 1a, the offline stage builds a proximity graph over the vector database, where each vector is a vertex and edges connect nearby vertices. During online search, the query traverses this graph and visits only a small subset of promising vertices instead of scanning the full database. Figure 1c summarizes the search procedure. In each iteration, the algorithm expands the current frontier vertex, fetches its neighbors, evaluates their distances to the query, and updates the candidate and result sets. Thus, although GANNS avoids exhaustive comparison, it still relies on repeated query-to-candidate distance evaluation during traversal.

A Running Example. As shown in Figure 1b, the online search starts from the entry vertex V_0 . Expanding V_0 reveals its neighbors. Since the candidate-set width is two in this example, only the two vertices closest to the query Q , namely V_1 and V_3 , are retained as active candidates, while the remaining neighbor is discarded. Their distances also initialize the current result set. The search then follows a best-first strategy [34]

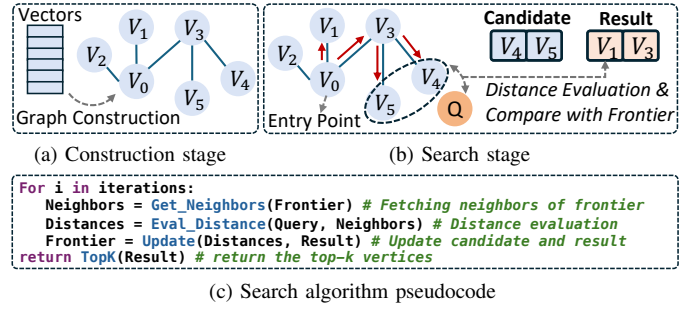


Fig. 1. Illustration of Graph-based ANNS.

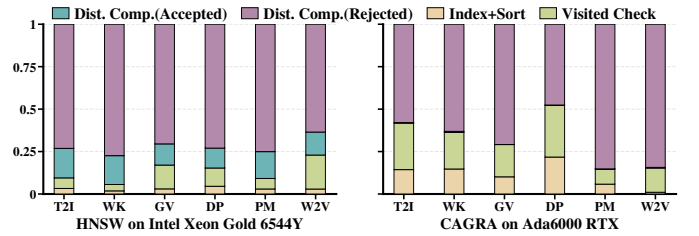


Fig. 2. Runtime breakdown of graph-based ANNS on CPU and GPU platforms. Distance computation is divided into accepted candidates that survive frontier update and rejected candidates that are visited but ultimately discarded.

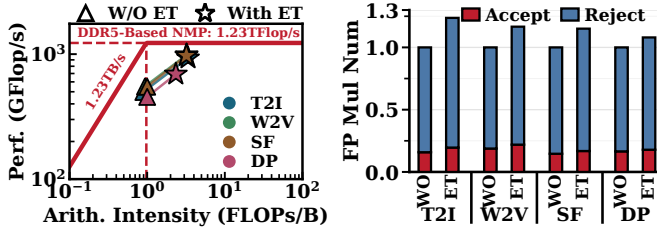
and expands the closer candidate, V_3 , which reveals two new neighbors, V_4 and V_5 . These newly discovered vertices are inserted into the candidate set, and their distances to Q are compared against the current worst result. In the illustrated state, the candidate set contains $\{V_4, V_5\}$, while the result set remains $\{V_1, V_3\}$. Any candidate that improves the current results is retained for further expansion.

B. Bottlenecks of Graph-based ANNS

When running GANNS on CPUs or GPUs, a large fraction of execution time is spent on query-to-candidate distance evaluation, including both vector fetch and arithmetic cost [19], [35]. To better understand this bottleneck, we profile representative GANNS implementations. As shown in Figure 2, distance evaluation dominates online search, accounting for over 75% of end-to-end runtime in hnsplib [8] on CPUs and around 68% in CAGRA [9] on GPUs. More importantly, a substantial fraction of this cost is spent on rejected candidates rather than useful survivors. This inefficiency stems from the execution pattern of graph traversal: each expansion fetches vector elements and evaluates the distances of the current candidate and its neighbors, but many of these visited candidates do not survive frontier update. Therefore, accelerating GANNS requires not only efficient distance evaluation, but also effective early termination to filter out non-promising candidates before incurring full distance cost.

C. Near-Memory Processing for GANNS

Near-memory processing (NMP) [10]–[13], [36], [37] is particularly attractive for GANNS because online search repeatedly fetches vectors from memory, making data movement



(a) NMP GANNS accelerators become compute-bound with ET (b) Hybrid partial-dimension/bit ET adds FP multiplications

Fig. 3. Existing NMP GANNS accelerator [16] remain bottlenecked by floating-point multiplication in both distance evaluation and pruning with hybrid partial-dimension/bit early termination (ET).

a bottleneck. Prior NMP-based GANNS accelerators [16] therefore commonly combine two complementary ideas: near-memory distance evaluation to reduce data movement, and early termination (ET) [18]–[20], [38], [39] to reduce unnecessary processing of ultimately discarded candidates.

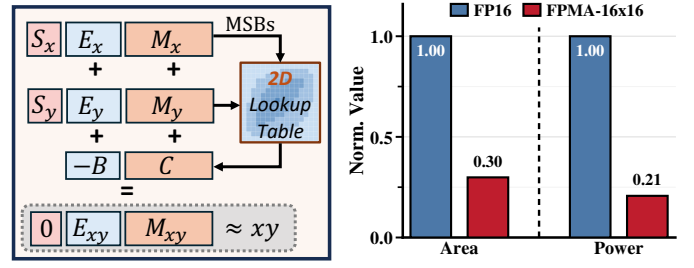
Tackling Memory Bottleneck with NMP. Prior NMP-based GANNS accelerators [16], [17] push distance evaluation toward the memory side, typically by distributing vectors across ranks and processing them in parallel near memory. This improves bandwidth efficiency and reduces the cost of vector fetch and distance evaluation for each visited candidate. However, NMP reduces the cost *per evaluated candidate* rather than the number of visited candidates themselves. Since graph traversal still explores many vertices that are ultimately discarded, near-memory execution alone cannot fully eliminate the inefficiency of candidate evaluation.

Reducing computation via early termination (ET). As shown in Figure 2, most candidates are ultimately discarded, so reducing unnecessary data fetches is a promising way to improve performance. Because each DRAM fetch returns part of vector data, ET incrementally estimates an informative lower bound after each fetch and uses it to determine whether a partially observed candidate can still remain competitive. Once the bound indicates that it cannot, the remaining fetches for that vector are terminated. This makes ET particularly attractive for NMP, where early rejecting directly avoids further memory accesses.

D. Limitations of Existing NMP-Based GANNS

Prior NMP-based GANNS accelerators reduce data movement, but they do not eliminate the dominant arithmetic cost. As shown in Figure 3a, their operating points already approach the compute roof, and ET pushes them further toward higher arithmetic intensity and deeper into the compute-bound regime across four datasets. The root cause is that costly FP multiplication remains in both distance evaluation and ET-based pruning.

• **Inefficiency #1: Multiplier-Intensive Distance Evaluation.** Even after distance evaluation is pushed closer to memory, the evaluation path remains multiplier-intensive. Under ℓ_2 -distance, distance evaluation must compute and accumulate per-dimension squared-difference terms, making FP multiplication a major component. This cost is further amplified



(a) Prior FPMA architecture with 2D (b) Area and power comparison against a DW [40] FP16 multiplier compensation table

Fig. 4. Prior FPMA design and hardware efficiency.

in NMP-based GANNS, where the same compute path is replicated across many ranks under tight area and power budgets. For example, ANSMET [16] processes each fetched 64 B block with a 16-wide multiplier-and-adder array in a DRAM rank. Therefore, while NMP alleviates data movement overhead, the distance evaluation path still incurs substantial multiplier cost, which limits overall efficiency.

• **Inefficiency #2: Multiplier-Intensive ET.** ET reduces unnecessary data fetches, but it is not free. Because ET is invoked for every visited candidate across multiple data-fetch rounds, its cumulative compute cost can be substantial. The state-of-the-art NMP GANNS design [16] employs hybrid partial-dimension/bit ET, which requires $\frac{512}{\text{Bitstep}}$ FP multiplications per 64 B DRAM fetch, where *Bitstep* denotes the number of bits fetched per dimension in one access. As shown in Figure 3b, enabling this ET consistently increases the total number of FP multiplications across all evaluated settings. The benefit of ET therefore depends not only on reducing data movement, but also on keeping multiplication overhead low; otherwise, overall performance becomes sensitive to multiplier overhead.

III. TOWARDS LIGHTWEIGHT NMP-BASED GANNS

Our key idea is that the search quality of GANNS depends on retrieval outcomes rather than exact intermediate distance values. This tolerance allows prior NMP-based designs to reduce rank-side arithmetic cost in both distance evaluation and ET.

A. Insights on Multiplier-Free GANNS

We derive two insights toward lightweight, multiplier-free NMP-based GANNS.

• **Insight #1: Distance evaluation can be made multiplier-free with FPMA.** GANNS is tolerant to moderate arithmetic imprecision because search quality depends primarily on preserving the ranking and retention of promising candidates during traversal, rather than on the exact numerical value of every query-to-candidate distance. This suggests that exact FP multiplication can be relaxed as long as approximate distance computation preserves retrieval-relevant ordering. This observation motivates us to explore *floating-point multiplication approximation* (FPMA) [21], [22], [41], [42] for GANNS. FPMA

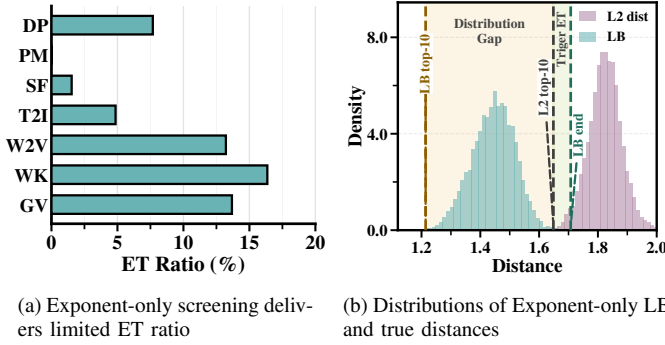


Fig. 5. Exponent-only bound is too loose to achieve effective screening.

replaces FP multiplication with low-cost integer-domain additions by approximating multiplication in the log domain, while retaining standard FP encoding. Prior work further improves FPMA accuracy with compensation structures, trading some additional hardware cost for better approximation quality. As shown in Figure 4, compared with a DW FP16 multiplier [40], FPMA16 with a 16×16 compensation table achieves $3\times$ area reduction and nearly $5\times$ power reduction, indicating strong hardware-cost potential for GANNS.

• **Insight #2: ET can be made multiplier-free with exponent-domain ordering.** ET in GANNS is fundamentally a *screening* problem: its goal is not to compute an accurate distance, but to decide whether a candidate is clearly non-promising before full-distance evaluation. For this purpose, the key requirement is not a precise numerical estimate, but a cheap signal that preserves enough ordering information to distinguish weak candidates from competitive ones. This creates an opportunity to derive the screening signal directly from sign and exponent metadata. Because the exponent captures order-of-magnitude information, coarse competitiveness can be estimated in the exponent domain without FP multiplication. For example, the scale of a product can be approximated as

$$\log_2 |xy| \approx (E_x - B) + (E_y - B), \quad (2)$$

which requires only integer-domain additions. Applied to ET, this suggests that sign/exponent can provide a much cheaper screening signal than conventional partial-distance updates, while still preserving useful ordering information for pruning.

B. Challenges of Multiplier-Free NMP GANNS

While the above insights suggest the promise of multiplier-free NMP-based GANNS, turning them into a practical rank-side design remains non-trivial.

• **Challenge #1: How to preserve retrieval fidelity with low-overhead multiplier-free distance evaluation.** Prior FPMA designs are not directly suitable for GANNS. In nearest-neighbor search, distance errors can perturb candidate ordering and thus degrade recall. However, improving approximation fidelity usually requires larger error compensation structures, which increase area and power overhead. This creates a key

challenge for rank-side NMP: how to achieve the fidelity required by GANNS without losing the hardware-efficiency benefit of multiplier-free computation.

• **Challenge #2: How to make exponent-domain information useful for ET.** Exponent-only estimates are not directly usable as drop-in replacements for conventional lower bounds, because they are often too loose to trigger effective threshold-based pruning. The challenge is therefore not to make exponent-domain signals numerically tight, but to redesign ET around the information they preserve reliably, namely coarse candidate ordering.

• **Challenge #3: How to translate lightweight screening into real system-level gains.** Lightweight screening alone does not automatically improve overall search efficiency. To avoid unnecessary full-vector retrieval, useful screening information must be exposed early in the access flow, while surviving candidates must still proceed to full distance evaluation. The challenge is therefore to co-design the NMP architecture, data layout, and access policy so that lightweight screening translates into real memory-traffic reduction and end-to-end speedup.

IV. MFNNS SOLUTION OVERVIEW

To address the above challenges, we propose **MFNNS**, a lightweight NMP-based GANNS accelerator that removes multiplier overhead from both the distance-evaluation path and the ET path, while co-designing the memory system to expose and exploit pruning opportunities. MFNNS consists of three tightly coupled components:

• **Square-specialized multiplier-free distance evaluation with FPMA.** MFNNS introduces a GANNS-oriented multiplier-free distance engine based on FPMA, but does not directly adopt prior generic designs. Instead, it specializes the approximation for squared-difference computation in ℓ_2 -distance evaluation, converting generic multiplication-oriented compensation into a more compact square-specialized form. This enables MFNNS to better balance retrieval fidelity and hardware overhead for GANNS.

• **Order-preserving multiplier-free ET.** MFNNS derives a lightweight screening signal directly from sign/exponent metadata using only integer-domain additions. Rather than treating exponent-only estimates as conventional threshold-style lower bounds, MFNNS redesigns ET as order-preserving screening: candidates that are clearly non-promising are filtered early based on coarse ordering information, while potentially competitive candidates are retained for full-distance evaluation.

• **ET-aware NMP architecture and memory organization.** MFNNS further co-designs the NMP architecture, data layout, and access policy so that lightweight ET leads to real system-level benefit. In particular, MFNNS exposes low-cost metadata early, separates screening from full-vector retrieval, and organizes memory accesses to improve row locality and reduce unnecessary payload fetches. This allows algorithmic pruning opportunities to be converted into tangible memory-traffic reduction and performance improvement.

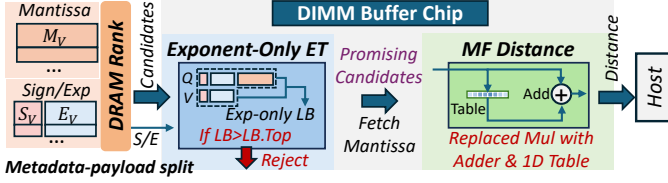


Fig. 6. MFNNS overview.

Figure 6 illustrates the overall workflow of MFNNS. For each visited candidate, MFNNS first fetches lightweight metadata and performs exponent-domain screening. Candidates identified as clearly non-promising are filtered early, avoiding unnecessary payload accesses and distance computation. The remaining candidates then proceed to multiplier-free distance evaluation using the square-specialized FPMA engine. Through this three-way co-design across distance evaluation, ET, and memory organization, MFNNS reduces rank-side multiplier overhead while improving both search efficiency and system efficiency.

V. SPECIALIZED FPMA FOR GANNS DISTANCE COMPUTATION

A. FPMA Preliminaries

Prior FPMA approximates FP multiplication by converting magnitude multiplication into addition in the log domain [21], [22], [41], [42]. For a normalized floating-point value $x = (-1)^{S_x} \cdot 2^{E_x - B} \cdot (1 + M_x)$, prior FPMA uses the approximation

$$\log_2(1 + M_x) \approx M_x \quad (3)$$

to represent multiplication with low-cost exponent/mantissa additions. For a product $r = x \cdot y$, this yields

$$R = X + Y - B, \quad (4)$$

where $X = E_x + M_x$, $Y = E_y + M_y$, and $R = E_r + M_r$ denote fixed-point concatenations of exponent and mantissa fields. Prior work [21] further introduces compensation tables to reduce approximation error, trading additional hardware overhead for higher accuracy.

However, FPMA is useful only if it preserves the candidate ordering needed to maintain recall of GANNS. The following analysis first bounds the numerical fidelity required by recall, then shows why generic FPMA remains limited by a large compensation structure, and derives a square-specialized distance primitive that aligns the approximation hardware with the dominant arithmetic pattern of GANNS. Finally, Section V-E analyzes how bounded FPMA error affects GANNS traversal.

B. Accuracy Requirements

We evaluate the impact of numerical variations on GANNS performance using HNSW [8], a representative GANNS algorithm, under experimental setup detailed in Section VIII-A3. Our goal is to identify the minimum numerical fidelity required by recall, and then examine whether FPMA can satisfy this requirement efficiently.

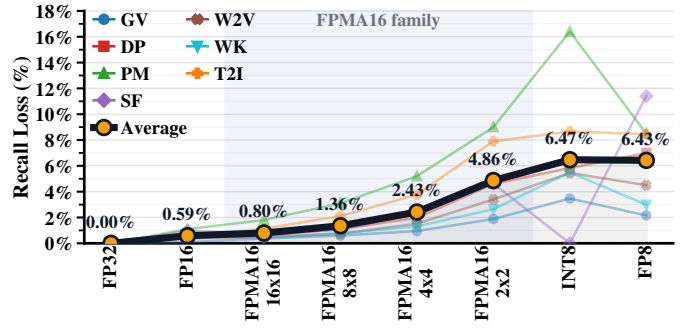


Fig. 7. Robustness of GANNS to reduced precision and approximate multiplication.

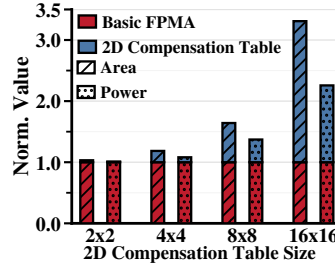


Fig. 8. Hardware cost of generic FPMA. Large 2D compensation tables dominate the area and power.

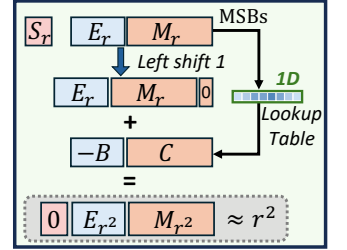


Fig. 9. Square specialization of FPMA. Compensation collapses from a 2D table to a 1D lookup.

Robustness to Reduced Precision. We first evaluate the sensitivity of GANNS to reduced numerical precision by quantizing vectors during search stage to several formats, using FP32 as baseline and comparing against FP16, FP8, and INT8. As shown in Figure 7, FP8 and INT8 often incur substantial recall degradation, whereas FP16 and higher-precision formats maintain stable recall. This establishes a clear lower bound on the numerical design space: except for datasets that are naturally amenable to lower-precision representations (e.g., SF), the distance-evaluation datapath should retain at least FP16-class dynamic range to avoid significant recall loss.

Compensation Requirements for Generic FPMA. This FP16-class requirement naturally raises the next question: can generic FPMA provide comparable retrieval quality at lower hardware cost? As shown in Figure 7, the answer is partially yes. FPMA16 maintains near-FP16 recall only when equipped with a relatively large 2D compensation table. In particular, a 16×16 table preserves recall close to FP16, whereas smaller tables such as FPMA16- 4×4 and FPMA16- 2×2 incur substantial accuracy loss. Thus, although generic FPMA can in principle meet the fidelity requirement of GANNS, doing so still requires nontrivial compensation hardware.

Together, these results reveal the key tension in GANNS distance computation: preserving recall requires FP16-class numerical fidelity, yet generic FPMA achieves this only with a relatively large 2D compensation table.

C. Motivation for Square-Specialized FPMA

Although FPMA16 with a 16×16 compensation table significantly reduces multiplication cost, the 2D table still dominates the design, occupying more than half of the FPMA multiplier area in the 16×16 configuration, as shown in Figure 8. This is inefficient for GANNS distance evaluation, whose dominant arithmetic pattern is per-dimension squaring of the residual term $(q_i - v_i)^2$ rather than general multiplication between two independent operands.

Let $r_i = q_i - v_i$ denote the normalized floating-point residual in each dimension. In generic FPMA (Figure 4a), compensation depends on the mantissa pair of the two inputs, so the compensation table is indexed by the leading MSBs of both mantissas to cover arbitrary operand combinations. For r_i^2 , by contrast, both inputs are identical and share the same mantissa M_r . Compensation therefore collapses from a two-operand problem to a one-operand one, and the lookup needs to be indexed only by the leading MSBs of M_r .

As a result, squaring requires only the diagonal entries of the original 2D compensation table. The compensation path can thus be reduced to a compact 1D lookup indexed by the leading MSBs of M_r . This observation naturally leads to a square-specialized FPMA design: by aligning the compensation structure with the square-dominated computation pattern of GANNS distance evaluation, MFNNS avoids the cost of a generic 2D table while preserving the accuracy benefits of compensation.

D. Square-Specialized FPMA Primitive

Based on the above observation, we construct a square-specialized FPMA primitive for per-dimension GANNS distance evaluation. As shown in Figure 9, the computation first forms the residual $r_i = q_i - v_i$ through FP subtraction, and then specializes the approximation path to the square structure of r_i^2 .

Instead of performing a general two-operand logarithmic sum, the square unit exploits the fact that both operands are identical. Specifically, it computes $M_r + M_r$ using a one-bit left shift by appending a trailing zero to M_r , and then applies a 1D compensation lookup indexed by the leading MSBs of M_r . As a result, the generic FPMA mantissa-multiplication and 2D-compensation path is replaced by a much simpler square-specialized datapath.

Although the architecture is optimized for ℓ_2 -distance, it also supports inner product (IP) and cosine similarity without structural modification. Specifically, IP, defined as $d^{(I)} = -\sum_{i=1}^D q_i v_i$, can be reduced to an ℓ_2 -distance computation [23]–[25]:

$$-\sum_{i=1}^D q_i v_i = \|Q - V\|_2^2 / 2 - (\|Q\|_2^2 + \|V\|_2^2) / 2. \quad (5)$$

Given a query, the query norm $\|Q\|_2^2$ is a runtime constant computed once by the host CPU, while the database vector norms $\|V\|_2^2$ are precomputed offline. Therefore, the accelerator only needs to compute the core $\|Q - V\|_2^2$ term. Similarly,

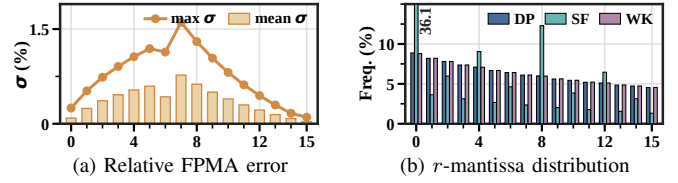


Fig. 10. Square-specialized FPMA error and r -mantissa distribution. X-axis denotes the MSB of r -mantissa.

cosine similarity, defined as $d^{(C)} = d^{(I)} / \sqrt{\|Q\|_2^2 \|V\|_2^2}$, can be supported either by offloading the final scalar division to the host CPU or by pre-normalizing the database vectors offline [16]. Consequently, an ℓ_2 -based square-specialized datapath is sufficient to support diverse distance metrics without introducing additional multiplier overhead.

E. FPMA error propagation in GANNS

Square-specialized FPMA introduces bounded distance error, whose effect on GANNS traversal is limited to narrow cases where a boundary comparison is reversed. In traversal, distances only rank candidates for queue updates; therefore, FPMA can affect candidate membership only if it flips the comparison between an incoming vertex and the current worst candidate. In such a case, a less-promising vertex may be retained. For a per-dimension FP16 residual $r = q_j - v_j$, we define the relative error of compensated FPMA as σ and $|\tilde{r}^2 - r^2| \leq \sigma r^2$. Since this error is determined by the FP16 mantissa and the down-sampled compensation table [21], [22], we can exhaustively enumerate all FP16 mantissa values. With the 16-entry square compensation table, the worst-case error is $\sigma_{\max} \approx 1.6\%$ (Figure 10a). Thus, for ℓ_2 -distance,

$$|\widetilde{Dist}(Q, V) - Dist(Q, V)| \leq \sigma Dist(Q, V). \quad (6)$$

During graph traversal, consider the comparison between V_b , the current worst vertex in the candidate set, and V_c , a vertex being evaluated for insertion. If the exact distances satisfy $Dist(Q, V_b) > Dist(Q, V_c)$, FPMA preserves this ordering when

$$Dist(Q, V_b) - Dist(Q, V_c) > \sigma (Dist(Q, V_b) + Dist(Q, V_c)). \quad (7)$$

Equivalently, $\frac{Dist(Q, V_b)}{Dist(Q, V_c)} > \frac{1+\sigma}{1-\sigma} \approx 1.03$. Thus, FPMA cannot change the candidate set update when the boundary distance gap is larger than about 3% of $Dist(Q, V_c)$. Figure 10b shows that MSB of r 's mantissa is relatively evenly distributed across datasets except SF, and the average relative square error is only $\bar{\sigma} \approx 0.36\%$, corresponding to 0.73% of $Dist(Q, V_c)$. This condition provides an a priori sufficient criterion for preserving boundary comparisons when applying FPMA to GANNS. We further analyze this effect in Section VIII-C2.

VI. MULTIPLIER-FREE EARLY TERMINATION

To reduce the cost of early termination (ET), MFNNS first explores the most direct multiplier-free strategy: constructing a lower bound using only sign/exponent metadata. However, although this exponent-only bound is cheap to compute, it is too

loose in magnitude to serve as an effective pruning threshold in conventional ET. MFNNS therefore reinterprets exponent-only bounds not as tight thresholds, but as cheap ordering signals, and builds an order-preserving ET mechanism around this property.

A. Exponent-Only Lower Bounds

We begin with the most direct multiplier-free ET design: constructing a lower bound using only sign/exponent metadata, so that mantissa processing and FP multiplication can be avoided during screening. This yields a very cheap screening signal, but its usefulness depends not only on computation cost, but also on how tightly the resulting bound tracks the true candidate distance.

For a candidate vector V , the lower bound is an intermediate result computed from partially fetched data of V and query Q . Once this lower bound exceeds the current worst result, the processing of V can be terminated early. We denote this multiplier-free lower bound as $LB(Q, V)$. For ℓ_2 -distance,

$$Dist(Q, V) = \|Q - V\|_2^2 = \|Q\|_2^2 + \|V\|_2^2 - 2 \sum_{i=1}^D q_i v_i \quad (8)$$

where $\|Q\|_2^2$ is constant across all candidates and $\|V\|_2^2$ can be precomputed and stored for each database vector. Therefore, lower bound estimation reduces to finding the lower bound of $-2 \sum_{i=1}^D q_i v_i$. This is the same as in inner-product and cosine-based search, where the lower bound estimation involves estimating $-\sum_{i=1}^D q_i v_i$.

This reformulation allows the distance to be analyzed through the sum of $-q_i v_i$. Using the query value q_i and only the *sign* and *exponent* fields of v_i , we construct a conservative bound that is safe for pruning and does not require the mantissa of v_i . For the two possible sign relations between q_i and v_i , the bound is:

- 1) **Case 1** where $sign(q_i) \neq sign(v_i)$

$$-q_i v_i = |q_i| \cdot (1 + M_{v_i}) \cdot 2^{E_{v_i} - B} \geq |q_i| \cdot 2^{E_{v_i} - B}. \quad (9)$$

- 2) **Case 2** where $sign(q_i) = sign(v_i)$

$$-q_i v_i = -|q_i| \cdot (1 + M_{v_i}) \cdot 2^{E_{v_i} - B} \geq -|q_i| \cdot 2^{E_{v_i} - B + 1}. \quad (10)$$

In both cases, the bound computation reduces to $|q_i| \cdot 2^n$. Since multiplication by 2^n only shifts the exponent of $|q_i|$, this operation can be implemented as a fixed-point addition between n and the exponent of $|q_i|$, instead of a mantissa multiplication. The hardware therefore realizes the bound through exponent updates and fixed-point additions, avoiding a mantissa multiplier. Summing the per-dimension bounds over all dimensions yields a multiplier-free term B_{init} . Therefore, for ℓ_2 distance, $LB(Q, V) = \|Q\|_2^2 + \|V\|_2^2 + 2B_{init}$; for inner-product and cosine-based search, $LB(Q, V) = B_{init}$.

Although this bound is inexpensive to compute, its absolute tightness is limited. Across seven datasets, the best ET rate is only about 20% in Figure 5a, corresponding to only about 10%

DRAM-traffic reduction. Figure 5b explains why: the lower-bound distribution (green) often remains far from the true-distance distribution (pink), leaving substantial slack. Because the bound is too loose in magnitude, magnitude-based ET rarely rejects candidates effectively. Therefore, the main value of exponent-only bounds lies not in tight thresholding, but in the coarse ordering information they preserve.

B. Order-Preserving Screening with LBQueue

Order-preserving observation. Although exponent-only lower bounds are too loose in magnitude for effective conventional ET, we observe that they still preserve the relative ordering of distances for most candidate pairs. The desired relation is

$$LB(Q, V_i) < LB(Q, V_j) \Rightarrow Dist(Q, V_i) < Dist(Q, V_j). \quad (11)$$

As shown in Table I, across 1000 queries on seven datasets, more than 84% of candidate pairs satisfy this ordering relation. The reason is that the exponent captures the dominant scale of each per-dimension contribution. Although mantissa-level details are omitted, the exponent alone still reflects the coarse magnitude of the distance contribution in each dimension.

This observation changes how exponent-only lower bounds should be used. Since their absolute values are too loose to serve as effective pruning thresholds, they should not be treated as conventional lower bounds for magnitude-based ET. Instead, their value lies in preserving coarse candidate ordering at low cost. This suggests an order-preserving screening criterion: use exponent-only lower bounds as cheap ranking proxies, and reserve full-distance evaluation only for candidates that remain competitive under this coarse ordering signal.

LBQueue-based early termination. Based on the above criterion, MFNNS replaces magnitude-based pruning with order-based pruning. The key idea is to use the ordering information preserved by the exponent-only lower-bound $LB(Q, V)$ to determine whether a vector V is competitive. To this end, MFNNS maintains a lower-bound queue, *LBQueue*, in parallel with the distance queue, *DistQueue*. *LBQueue* tracks the coarse ordering of currently retained candidates under the exponent-only lower-bound, while *DistQueue* preserves the final ranking based on the full multiplier-free distance. For candidate V , MFNNS first computes $LB(Q, V)$ using signs and exponents only. If $LB(Q, V) > LBQueue.top()$, the candidate is discarded immediately, because its coarse bound is already worse than the weakest lower-bound rank among the current survivors. Otherwise, the candidate passes screening: MFNNS fetches the mantissa payload, computes the full multiplier-free distance, and compares

TABLE I
ORDER-PRESERVING RATIOS ACROSS DATASETS.

Dataset	T2I	WK	GV	DP	PM	W2V	SF
Ratio	93.72%	90.80%	84.90%	91.67%	91.88%	94.31%	96.18%

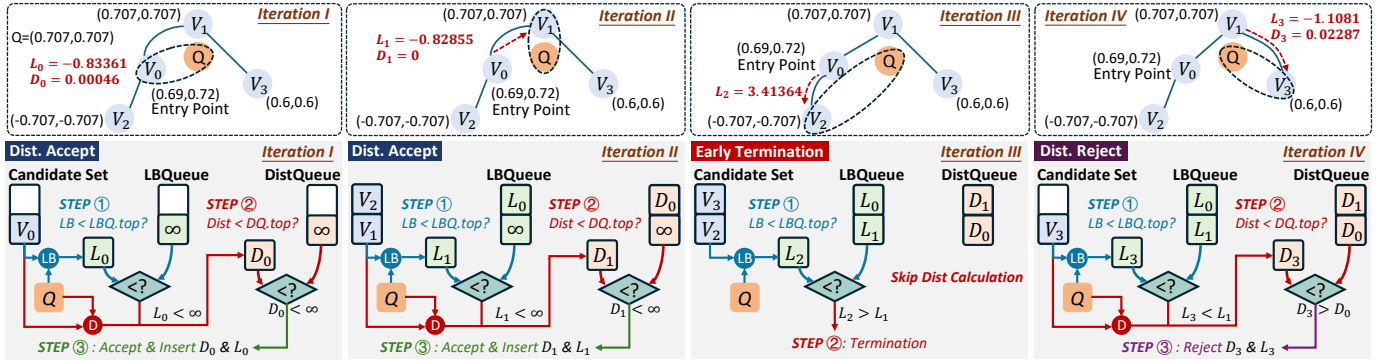


Fig. 11. LBQueue-based early termination. Showing three outcomes: accept after screening and distance evaluation, reject directly after exponent-domain screening, and reject after passing screening but failing distance ranking.

it against $DistQueue.top()$. If the distance is smaller, the candidate is inserted into $DistQueue$, and its lower-bound is inserted into $LBQueue$; otherwise, it is rejected.

Figure 11 illustrates the control flow with a four-iteration example. Iterations I and II follow the accept path. In Iteration I, the queues are empty, so the lower bound (L_0) between Q and V_0 passes the screen trivially, the algorithm evaluates distance between Q and V_0 to get D_0 , and L_0 and D_0 are inserted in $LBQueue$ and $DistQueue$, respectively. Iteration II shows the same path after the computation of V_0 has finished: the next candidate V_1 again passes Step ①, enters the full-distance stage, and inserts L_1 and D_1 . Iteration III shows the evaluation of V_2 . The lower bound between Q and V_2 is $L_2 = 3.41364$, which is larger than L_1 , the top of $LBQueue$; V_2 is therefore rejected after Step ①, and skips full-distance computation. Iteration IV shows the different case. The lower bound between Q and V_3 is L_3 , and L_3 is less than L_1 , the top of $LBQueue$. Therefore, the candidate remains acceptable under coarse exponent screening and need to evaluate distance D_3 (Step ②). In this case, D_3 is larger than D_0 , the current top of $DistQueue$, so it fails the distance test and is rejected at Step ③. Neither L_3 nor D_3 is stored. Across the four columns, the figure makes the division of labor explicit: $LBQueue$ removes obviously uncompetitive candidates cheaply, while $DistQueue$ preserves the final ranking for the cases that are not terminated.

This design does not require exponent-only lower bounds to be numerically tight. As long as they preserve coarse ordering sufficiently well, $LBQueue$ can filter out many non-promising candidates before mantissa fetch and full-distance computation. Moreover, as order preservation weakens, pruning aggressiveness can be tuned by adjusting $LBQueue.size$, i.e., the length of $LBQueue$, thus providing an explicit trade-off between efficiency and accuracy.

VII. HARDWARE ARCHITECTURE

The MFNNS hardware supports the lower-bound and distance evaluation, while remaining GANNS search operations, such as graph index traversal, are executed on the host. Figure 12a illustrates the hardware architecture of MFNNS.

Specifically, each rank integrates a DIMM-level module into the buffer chip (Figure 12c), along with dedicated hardware for square computation (Figure 12d), and corresponding instruction support for offloading lower-bound and distance evaluation (Figure 12e).

A. Hardware Modules and Execution Flow

1) **DIMM-level Processing Engine (DPE)**: The DPE (Figure 12c) consists of the control part and the compute part to offload lower bound and full distance evaluations. The control part contains an instruction queue, a parser, and a memory controller; it parses host instructions to extract DRAM addresses and operation fields, passing them to the memory controller, which issues ACT/RD/PRE sequences to fetch vector data. The compute part comprises square units, scale units, and an adder tree, which are used to calculate either the exponent-only lower bound or the full distance.

2) **Square and Scale Units**: As shown in Figure 12d, Square Unit contains an FP16 adder and a 10-bit fixed-point adder. The FP adder and 10-bit adder perform the calculation of $(q_i - v_i)^2$, which forms $r = q_i - v_i$ through FP adder and then produces an approximate squared term using the 1D compensation table.

The Scale Unit comprises two 5-bit fixed-point adders and one shifter. One adder performs lower-bound estimation by summing the exponents of q_i and v_i . The other adder, together with the shifter, converts each squared term, or the exponent-only lower bound, into a fixed-point value. The unit stores scaling constants provided by the host instruction and scales each input by subtracting its encoded exponent from the selected constant and applying the resulting shift to the mantissa. The shifted mantissas are then accumulated across dimensions by a fixed-point adder tree.

The Square and Scale unit supports multiple data types. For fixed-point inputs, the graph index stores the leading-zero information during index construction to support the screening stage. This information acts as an exponent-like term for the fixed-point value [43]. During the full-distance evaluation stage, the square unit uses this leading-zero information to identify the leading one and then uses the subsequent bits to index the compensation table.

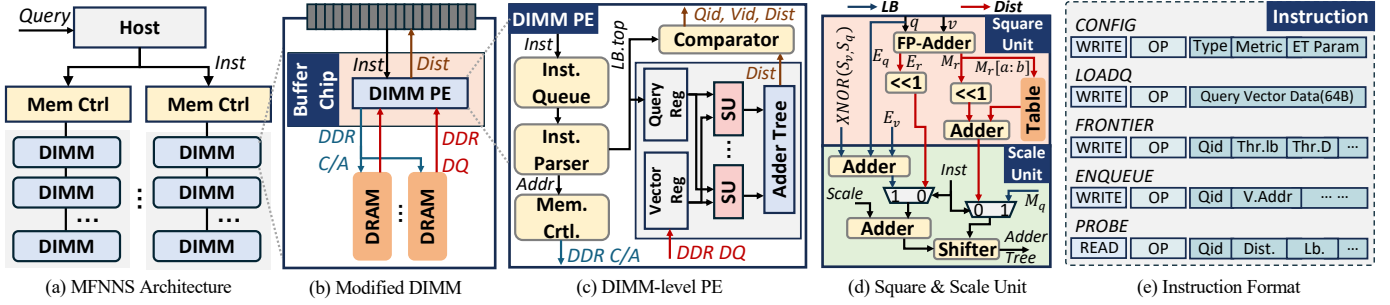


Fig. 12. MFNNS hardware architecture.

3) **Execution Flow:** MFNNS adopts a decoupled host-DIMM execution model. The host remains responsible for graph traversal, frontier maintenance, and polling-based result updates [16], [44], [45], while DPE executes rank-local screening and distance evaluation.

All commands except PROBE are issued as DDR writes, whereas PROBE is DDR read. The instruction set contains five commands. CONFIG initializes the DPE profile, including the datatype, vector dimension, metric mode, screening mode, compensation profile, and scale preset. LOADQ streams a query fragment into the query register of the target DPEs. During search, whenever the host expands a frontier vertex and forms a new candidate batch, it issues FRONTIER to update the current screening threshold T_{LB} and distance threshold T_D for that query, and then issues ENQUEUE to append candidate vector addresses into the DPE memory-control queue.

After a candidate is enqueued, the DPE executes a two-stage flow. In stage-one, the memory controller fetches the signs/exponents of the vector, and the Scale Unit computes the lower bound. If the result already exceeds the threshold T_{LB} , the candidate is discarded immediately and no mantissa access is issued. Otherwise, the same in-flight candidate proceeds to stage-two: the memory controller fetches the mantissa data, and the Square Unit, the Scale Unit, together with the adder tree, compute the distance. The DPE comparator then checks the final distance against the threshold T_D and records the completed result entry. Finally, the host periodically issues PROBE to read back completed pending results and merge them into the host-side result structures.

B. Data Layout and Optimization

Once ET screening and full distance evaluation consume different vector fields at different frequencies, memory layout and DRAM row policy should be co-designed accordingly.

Limitations of naive full-vector layout. Figure 13a shows the naive organization, where the sign/exponent fields and mantissa fields of each vector are stored together, and full vectors $V_1, \dots, V_l$ are packed sequentially into a row across the rank. This layout is inefficient for ET screening. The first ET stage only requires sign/exponent metadata, yet each access still fetches a full-vector row that includes mantissas. The resulting access granularity is therefore much larger than the screening working set.

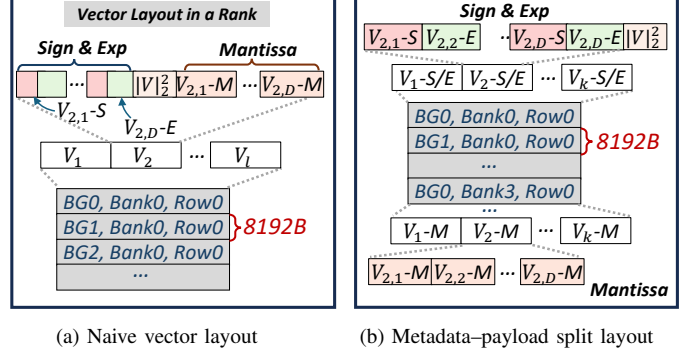


Fig. 13. ET-aware data layout optimization.

Metadata-payload split layout. Figure 13b shows the data layout of MFNNS. MFNNS separates each vector V_i into a compact metadata block $V_i\text{-S/E}$, formed by concatenating its per-dimension sign and exponent fields and the precomputed norm term $|V_i|^2$, and a payload region that stores the mantissa fields. Instead of storing only l full vectors in one rank row, the same 8192 B row can now pack sign/exponent metadata blocks $V_1\text{-S/E}, \dots, V_k\text{-S/E}$ for many more vectors, while the corresponding mantissa payload blocks $V_1\text{-M}, \dots, V_k\text{-M}$ are placed in separate rows. This organization aligns DRAM layout with MFNNS two-stage access pattern: ET screening touches only the compact sign/exponent metadata rows, whereas distance evaluation fetches mantissa payload rows only for candidates that are not terminated in the first stage. ET therefore reduces not only arithmetic work, but also unnecessary DRAM traffic. The split layout also makes the DRAM open-row policy more effective. Since a recently activated row is kept open rather than being immediately precharged, later accesses to the same row can hit in the row buffer without another activation. Graph traversal screens neighboring candidates in batches, so first-stage requests frequently target sign/exponent metadata entries that reside in the same activated metadata row. Keeping that row open amortizes row activation over many screenings and reduces row-buffer miss rates. In contrast, under the naive full-vector layout (Figure 13a), the same screening stream is diluted by unused mantissa data and is more likely to spill across multiple rows. The split layout thus converts screening locality into DRAM row locality.

VIII. EVALUATION

A. Experimental Setup

1) **Implementation:** We develop a cycle-accurate simulator for MFNNS by extending Ramulator 2.0 [46]. To estimate memory energy consumption, we leverage the DRAM power model integrated in Ramulator 2.0 [46]. For the SRAM in DPEs we obtain latency, energy, and area projections using CACTI [47] under a 28 nm technology node. We implement square unit, scale unit, instruction queue, and decoder in Verilog and synthesize the generated RTL targeting a TSMC 28 nm process and a 1 GHz clock frequency. Each DPE occupies 0.01 mm² with 16 square units, 16 scale units, an adder tree, an instruction queue, a decoder and 2KB SRAM, which is modest relative to the typical ~ 100 mm² area budget of commodity DIMM buffer chips [16], [48].

2) **Baselines:** We compare MFNNS against three baseline categories: CPU, GPU, and prior NMP-based designs under matched memory-side resources and matched recall.

- **CPU Designs:** HNSW [8] is evaluated on a CPU using the configuration shown in Table II.

- **GPU Designs:** The evaluated GPU designs include CAGRA [9], a SOTA GPU-based GANNS framework, and BANG [24], a GPU-based GANNS framework that supports billion-scale datasets by offloading vector data to CPU memory. It is important to note that billion-scale datasets exceed the memory capacity of a single GPU. Thus, we evaluate CAGRA on non-billion-scale datasets whose vectors and graph index fit in GPU memory. The detailed evaluation on billion-scale datasets is reported separately in Section VIII-B4. All GPU-side executions use an NVIDIA RTX 6000 Ada GPU. For BANG, the CPU-side component uses the configuration shown in Table II.

- **NMP Designs:** For prior NMP comparison, we use ANSMET [16]. We use the hybrid partial-dimension/bit early termination configuration as in ANSMET [16] and adopt 16-bit FP multipliers and adders for fair area and energy efficiency comparison. All NMP-based baselines are implemented using the same host CPU and DRAM configuration (Table II), and are provisioned with the same number of compute units (MACs, or Square Units in MFNNS) to ensure a fair comparison at matched recall. HNSW [8] is used as the representative GANNS algorithm for NMP-based designs. We evaluate six design points on this common substrate: **NMP-Base** offloads distance computation to memory; **NMP-FPMA** replaces the baseline distance engine with FPMA; **NMP-FPSA** uses the proposed square-specialized FPMA unit; **NMP-FPSA-ET** further adds the proposed early termination mechanism; **NMP-Base-ET** keeps the baseline FP distance engine but uses the proposed ET and memory layout; and **MFNNS** combines the proposed square-specialized FPMA, ET, and data-layout optimizations.

3) **Datasets & Metrics:** We evaluate the proposed architecture using nine commonly used diverse, real-world vector datasets summarized in Table III, covering a range of dimensionalities, index sizes, and application domains. These

TABLE II
SYSTEM CONFIGURATIONS USED IN EVALUATION.

Component	Configuration
Host CPU	Intel Xeon Gold 6544Y 16-Core, 3.6–4.1 GHz 48 kB L1-D, 32 kB L1-I, 2 MB L2, 45 MB shared LLC
Memory	DDR5-4800, 4 channels $\times$ 2 DIMMs $\times$ 4 ranks, 8 bank groups $\times$ 4 banks, RCD-CAS-RP: 40-40-40
DPEs	32 DIMM-level PE in total, one per rank, 1 GHz 32-entry Inst Queue, 1 Inst Decoder, 2KB SRAM 16 Square Units & 16 Scale Units, 16-input adder tree

TABLE III
ANNS DATASETS USED IN EVALUATION.

Domain	Dataset	Datatype	Distance	#Vectors	#Dims	#Queries	Size (GiB)
Text	Wiki (WK)	FP16	IP	1M	768	1000	1.43
	PubMed (PM)	FP16	L2	500K	768	1000	0.72
Image	DEEP1B (DP1B)	FP16	L2	1B	96	10000	178.81
	Txt2Img1B (T2I1B)	FP16	IP	1B	200	10000	372.53
	DEEP10M (DP)	FP16	L2	10M	96	1000	1.79
	Txt2Img1M (T2I)	FP16	IP	1M	200	1000	0.37
	SIFT1M (SF)	UINT8	L2	1M	128	1000	0.12
Words	GloVe (GV)	FP16	IP	2.2M	300	1000	1.23
	Word2Vec (W2V)	FP16	L2	1M	300	1000	0.56

TABLE IV
PREPROCESSING TIME OF DIFFERENT DATASETS.

Dataset	T2I	WK	GV	DP	PM	W2V	SF	DP1B	T2I1B
Preproc. time (s)	0.03	0.07	0.12	0.13	0.06	0.05	0.02	11.5	15.8
Graph constr. time (mins)	1.14	2.92	5.65	10.92	1.62	1.86	0.93	861	1070

datasets span use cases such as vision, natural language, and recommendation embeddings. To construct the HNSW index, we set *efConstruction* parameter to 100 and limit the maximum degree to 32 [49]. We tune the other parameter *efSearch* until the recall rate is over 90% [19]. Any small recall loss of our ET strategy can be compensated by increasing *efSearch* and *LBQueue.size* and reported results are taken at matched recall, which is further evaluated at Section VIII-C3. Table IV shows that the extra preprocessing required by MFNNS is negligible, especially compared with the unavoidable graph construction time: it reads the vectors, extracts sign/exponent metadata, packs mantissas into a separate region, and precomputes the per-vector norm term used by the IP/L2 reformulation.

B. End-to-End Evaluation

We compare MFNNS against CPU, CAGRA, BANG and ANSMET under matched compute resources and matched recall. The evaluation covers end-to-end throughput for $k = 5$, $k = 10$, and $k = 100$ on non-billion-scale datasets where CAGRA fits in GPU memory, recall-throughput tradeoffs on billion-scale datasets, and system-level area and energy efficiency for $k = 10$.

1) **Throughput Comparison:** Figure 14 compares the normalized query-per-second (QPS) of MFNNS against baselines across seven datasets. Under $k = 5$, $k = 10$, and $k = 100$. At 90% recall, MFNNS achieves the highest end-to-end throughput across all datasets, delivering a geometric-mean speedup

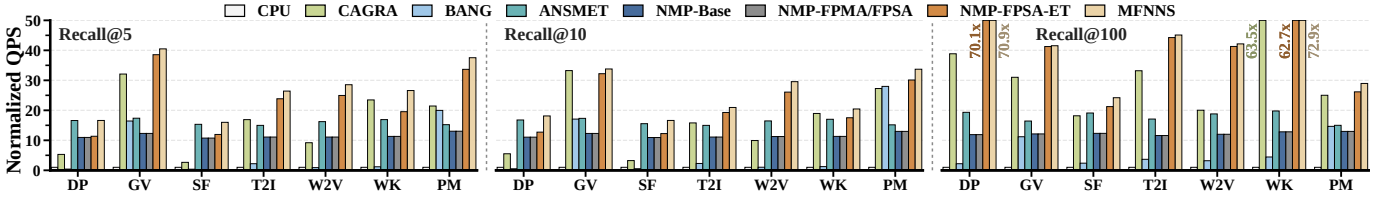


Fig. 14. Throughput comparison between MFNNS and the baselines with different datasets and result counts (k).

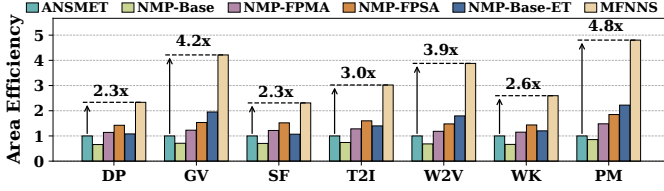


Fig. 15. Area efficiency comparison at Recall@10=90%.

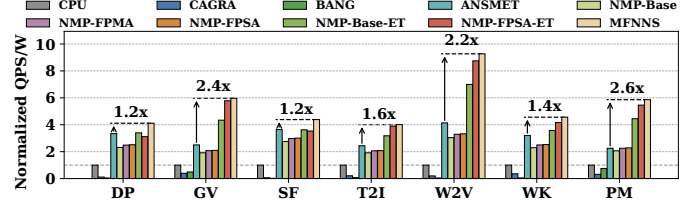


Fig. 16. Energy efficiency comparison at Recall@10=90%.

of $29.9\times$ over CPU, $10.8\times$ over BANG, $1.81\times$ over CAGRA, and $1.8\times$ over ANSMET.

The staged comparison from NMP-Base to NMP-FPMA, NMP-FPSA-ET, and MFNNS shows that this gain comes from cumulative bottleneck removal. We omit NMP-FPSA from this chain because it provides the same DPE compute performance, and thus the same end-to-end performance, as NMP-FPMA. NMP-FPMA is nearly identical to NMP-Base, differing only in the multiplier implementation. NMP-FPSA-ET improves throughput by $1.86\times$, $1.74\times$, and $3.32\times$ over NMP-FPMA at $k = 5$, $k = 10$, and $k = 100$, respectively. Layout optimization contributes an additional $1.22\times$, $1.18\times$, and $1.06\times$ by improving locality.

2) **Area Efficiency:** Figure 15 reports the normalized area efficiency of designs across seven datasets. MFNNS delivers a $3.18\times$ geometric-mean improvement in area efficiency over ANSMET, with per-dataset gains ranging from $2.31\times$ on SF to $4.8\times$ on PM. The staged comparison from NMP-Base to NMP-FPMA, NMP-FPSA, and MFNNS also shows monotonic gains across all datasets. NMP-FPMA and NMP-FPSA improve area efficiency by $1.73\times$ and $1.25\times$, respectively, over their preceding designs, while MFNNS adds a further $2.06\times$ gain over NMP-FPSA by incorporating ET. Compared with NMP-Base-ET, MFNNS achieves a dataset-independent $2.16\times$ area-efficiency gain for the smaller multiplier-free DPE area. Because NMP-FPSA, NMP-FPSA-ET, and MFNNS have the same DPE area, their improvements in area efficiency are identical to their throughput improvements.

3) **Energy Efficiency:** Each MFNNS DPE consumes only 3.8 mW. At the system level, MFNNS achieves a $2.28\times$ and $2.1\times$ efficiency improvement over NMP-Base and NMP-FPMA, respectively. Figure 16 reports system-level energy efficiency across seven datasets. In geometric mean, MFNNS improves power efficiency (QPS/W) by $5.23\times$ over CPU, about $26.06\times$ over CAGRA, about $56.97\times$ over BANG, and $1.74\times$ relative to ANSMET. For the SOTA NMP design, MFNNS sustains $1.2\times$ - $2.6\times$ higher QPS/W than ANSMET.

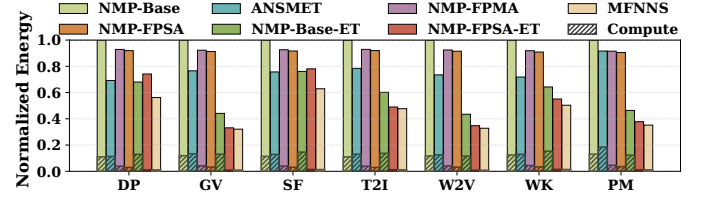


Fig. 17. System energy comparison between MFNNS and the baselines on different datasets. Normalized to NMP-Base.

The staged comparison from NMP-Base to NMP-FPMA, NMP-FPSA, NMP-Base-ET, NMP-FPSA-ET, and MFNNS further reveals that compute-only refinements provide incremental gains: NMP-FPMA and NMP-FPSA increase the geometric-mean QPS/W from $2.3\times$ to $2.49\times$ and $2.51\times$ over CPU, respectively. The larger improvement emerges after ET-based designs reduce memory access cost, with NMP-Base-ET, NMP-FPSA-ET, and MFNNS reaching $4.08\times$, $4.68\times$, and $5.23\times$ over CPU. ET provides the main system-level gain, while MFNNS hardware design provides the remaining 28.1% improvement over NMP-Base-ET.

Figure 17 shows the total energy breakdown for MFNNS, comparing with ANSMET and NMP-Base for $k = 10$. MFNNS reduces total energy by 58.91% relative to ANSMET and by 65.77% relative to NMP-Base. The staged ablation shows that most of this reduction comes from ET: NMP-Base-ET already lowers energy by 54.1% over NMP-Base. NMP-FPSA-ET further reduces it by 21.3% , indicating that compute-side overhead has been largely removed.

4) **Performance on Billion-Scale Datasets:** Figure 18 reports the recall-throughput curve on DP1B and T2I1B under both Recall@10 and Recall@100. BANG [24] is used for billion-scale GPU-side comparison because CAGRA cannot hold the full billion-scale indexes in GPU memory. Across the four designs, MFNNS improves throughput over ANSMET by $1.26\times$ - $1.31\times$ within the overlapping recall ranges. Specifically, the matched-recall speedups are $1.28\times$ on DP1B

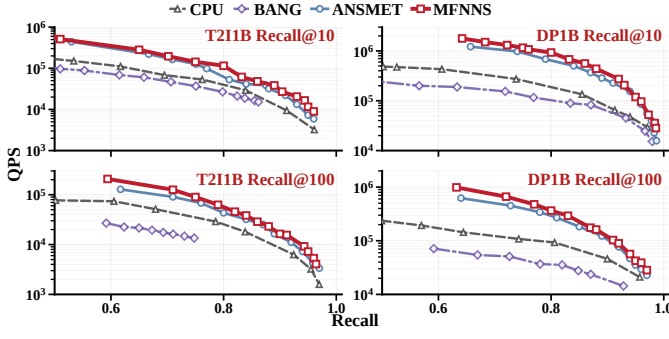


Fig. 18. Recall vs. QPS curves for billion-scale datasets.

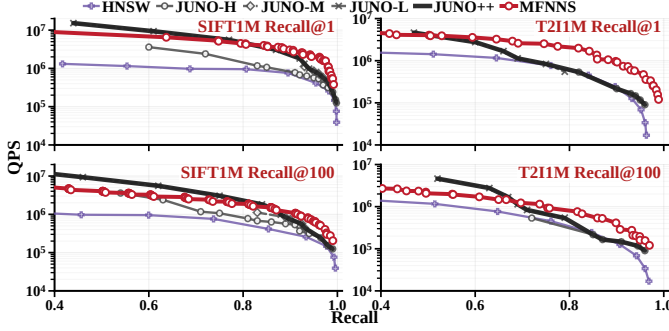


Fig. 19. Recall vs. QPS curves for MFNNS and JUNO++.

Recall@10, $1.26\times$ on T211B Recall@10, $1.31\times$ on DP11B Recall@100, and $1.29\times$ on T211B Recall@100. MFNNS also outperforms CPU by $2.41\times$ – $4.31\times$ and BANG by $4.54\times$ – $10.95\times$ over their overlapping recall ranges.

5) **Comparison with RT-Core-Accelerated ANNS:** Figure 19 compares MFNNS against the design-space frontier of JUNO++ [50], a SOTA ray-tracing-core-accelerated design for PQ-based ANNS. The JUNO++ data points are extracted from the results reported in the original paper. At a high-recall target of 0.95, MFNNS improves QPS by $1.50\times$ – $4.34\times$ over the high-recall JUNO-H configuration, with a geomean speedup of $2.39\times$. This advantage indicates that MFNNS sustains a stronger high-recall throughput. The benefit is not uniform across all operating points: part of JUNO-L achieves higher QPS in the low-recall region.

C. Performance Attribution Analysis

1) **Area and Power of DPEs:** As shown in Figures 20a and 20b, MFNNS reduces both area and power consumption within the DPE through two main optimizations. At the compute-unit level, its Square Units reduce area by 65.09% and 34.98%, and power by 66.67% and 25%, compared with the FP MACs used in ANSMET and a generic FPMA design, respectively. Replacing the FP adder tree with Scale Units further reduces both area and power. Together, these optimizations make the DPE both compact and power-efficient: an MFNNS DPE occupies only 0.01 mm^2 , improving area efficiency by $2.2\times$ and $1.2\times$ over ANSMET and NMP-FPMA, respectively. Such a small footprint aligns well with the tight

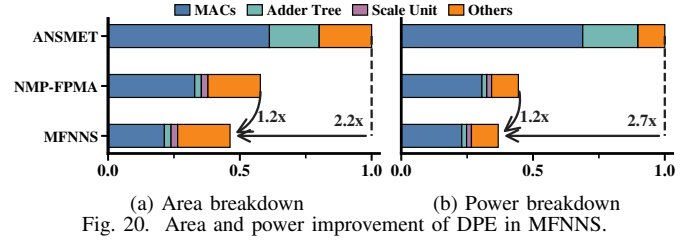


Fig. 20. Area and power improvement of DPE in MFNNS.

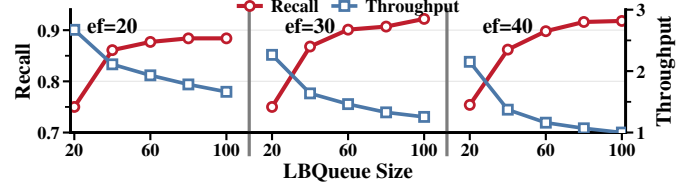


Fig. 21. Sensitivity of LBQueue size on T21.

area budget of commodity DIMM buffer chips [16]. Moreover, the DPE is not limited to GANNS, and can serve as near-memory extension for vector similarity evaluation, helping relieve the area and power overhead caused by replicated memory-side FP multipliers [36], [37].

2) **Algorithm Analysis:** Table V isolates the retrieval-quality impact and interaction of FPMA and ET under the same search budget. All variants use $k=10$, $efSearch=500$, and the size of *LBQueue* set to $efSearch$. Relative to FP16, FPMA reduces recall by an average loss of 0.77%. ET reduces recall by an average loss of 0.52%. Total average recall loss is 1.17% when enabling both.

The table also clarifies the interaction between FPMA and ET. ET computes a signal from sign/exponents, whereas FPMA is used only for distance evaluation after a candidate passes ET screening. Empirically, the ET ratios of FP16+ET and FPMA+ET are nearly identical across all datasets, differing by at most 0.33% and by only 0.06% on average. This confirms that FPMA has negligible direct impact on ET screening decisions.

The remaining interaction is indirect: FPMA can affect whether a vector updates *DistQueue*, which can in turn affect later *LBQueue* states. This coupling is limited in practice. Relative to FPMA-only, enabling *LBQueue* causes at most 0.41% additional recall loss on six of the seven datasets. At the same time, FPMA+ET sustains a high ET ratio of 68.45%–85.33%, indicating that *LBQueue* removes most full-distance evaluation, while FPMA reduces the cost of the remaining full-distance computations.

To connect the margin condition in Section V-E with real traversal behavior, we further log result-boundary comparisons during search. A risk update occurs when an exact-accepted candidate lies near enough to the boundary for FPMA to affect the update decision. Across traversal, only (1.42%) of updates are at risk, and the actual FPMA-induced candidate-update flip rate is only (0.236%).

3) **Parameter Sensitivity:** Figure 21 shows the *LBQueue* design space. A smaller *LBQueue* size means pruning more

TABLE V
RECALL AND EARLY TERMINATION RATIO ACROSS THE SEVEN DATASETS
UNDER $k=10$ AND $efSearch=500$.

Metric	Variant	T2I	WK	GV	DP	PM	W2V	SF
Recall	FP16	97.19%	96.91%	83.36%	99.13%	86.90%	97.32%	98.93%
	FPMA	95.58%	96.70%	82.85%	98.52%	84.90%	96.95%	98.85%
	FP16+ET	97.00%	96.60%	82.65%	98.86%	86.50%	97.16%	97.33%
	FPMA+ET	95.41%	96.39%	82.44%	98.23%	85.00%	96.83%	97.27%
ET Ratio	FP16+ET	79.64%	81.74%	85.34%	84.06%	79.68%	82.69%	68.46%
	FPMA+ET	79.64%	81.71%	85.33%	84.05%	79.35%	82.69%	68.45%
FPMA Influence	Risk Update	1.49%	0.89%	1.11%	0.49%	4.11%	0.93%	0.51%
	Flip Ratio	0.278%	0.104%	0.170%	0.124%	0.670%	0.141%	0.167%

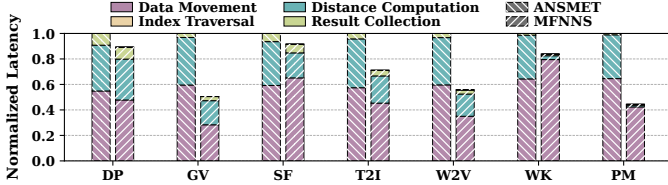


Fig. 22. Latency breakdown in ANSMET and MFNNS.

aggressively, which maximizes throughput but leaves recall loss, whereas larger queues progressively recover recall for consecutive screening. For $efSearch = 30$, a queue size of 86 already achieves a recall of 0.92, only 0.4% below the no-loss upper bound of 0.924. Similarly, under $efSearch = 40$, a queue size of 80 reaches 0.92 recall. These results show that MFNNS does not depend on a narrow parameter point; instead, it maintains a stable recall-throughput tradeoff across a practical range of queue sizes.

4) **Latency Breakdown:** Figure 22 breaks down the end-to-end query latency and attributes the overall speedup. Relative to ANSMET, MFNNS reduces total latency by 30.51% on average for $k = 10$, corresponding to $1.49\times$ speedup. Among all components, distance computation sees the largest reduction, decreasing by 55.38% on average, whereas index traversal and polling-based result collection, which are mainly handled by the host CPU, remain nearly unchanged. Specifically, MFNNS improves the non-overlappable portion of the search pipeline: FPMA and exponent-only screening jointly reduce the compute workload that cannot be hidden by memory access. Moreover, the smaller reduction in data movement, in turn, reflects the benefit of the ET condition.

5) **Memory Access Optimization:** Figure 23 shows that MFNNS improves the memory behavior by matching DRAM layout to the asymmetric two-stage access pattern of ET and full-distance calculation. By separating compact sign/exponent metadata from mantissa payloads, MFNNS packs many more metadata blocks into each 8192B row and allows the first ET stage to repeatedly access dense metadata rows. This organization makes the open-row reuse more effective and reduces row-buffer misses that would otherwise arise under the naive full-vector layout. Across seven datasets, MFNNS lowers row miss per read by 81.51% in geometric mean relative to NMP-FPSA-ET and by 56.70% relative to ANSMET. These results show that MFNNS converts ET-induced screening locality into actual DRAM row locality.

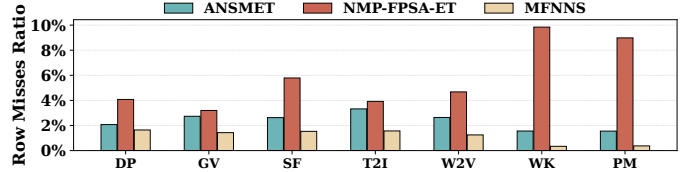


Fig. 23. Memory access optimization of MFNNS.

IX. RELATED WORK

Early Termination. Existing ET techniques prune candidates at the vector, dimension, or bit/partial-distance level. Vector-level methods [18], [20], [38], [39], [51] require learned predictors or offline auxiliary structures, making them less friendly to dynamic updates. Dimension-level and bit/partial-distance-level methods either provide limited termination efficiency or introduce repeated FP multiply-accumulate operations for partial-distance updates.

Hardware-based GANNS Acceleration. CAGRA [9] optimizes GANNS by exploiting massive parallelism on GPU. DF-GAS [52] improves ANNS throughput through customized datapaths and parallel graph search on FPGA. DiskANN [53] and PipeANN [54] reduce DRAM capacity pressure by placing billion-scale datasets on SSDs. Near-SSD designs, including VStore [55], NDSearch [56], SmartANNS [57], REIS [58], and ICE [59], further move part of the search computation closer to storage, but their effective bandwidth and latency remain limited by the SSD interface compared with near-memory processing. HeterRAG [17] accelerates RAG retrieval by offloading distance evaluation to DRAM, while ANSMET [16] incorporates ET for GANNS. However, its ET implementation relies on multiplication-heavy partial-distance updates, limiting end-to-end efficiency.

X. CONCLUSION

This paper presented **MFNNS**, a multiplier-free near-memory accelerator for GANNS that addresses the FP multiplication bottleneck in existing NMP designs. MFNNS eliminates FP multipliers from distance evaluation and ET through square-specialized FPMA and exponent-domain order-preserving screening, while co-designing the NMP architecture and data layout to reduce data accesses and overall search cost. At 90% recall, MFNNS achieves average speedups of $29.9\times$ and $1.81\times$ over the CPU and GPU baselines, respectively, with corresponding energy-efficiency gains of $5.23\times$ and $26.06\times$. Compared with the state-of-the-art NMP accelerator, MFNNS delivers $1.8\times$ higher throughput, $3.18\times$ better area efficiency, and $1.74\times$ higher energy efficiency.

ACKNOWLEDGEMENTS

This work is supported by National Key Research and Development Program of China (No. 2024YFB4504200) and the Guangzhou-HKUST(GZ) Joint Funding Program (No. 2025A03J3568). We also thank the AMD Heterogeneous Accelerated Compute Cluster (HACC) Program [60] for providing access to hardware resources.

APPENDIX A

ARTIFACT APPENDIX

A. Artifact Abstract

This artifact contains the source code and automated workflow for evaluating MFNNS, a multiplier-free near-memory accelerator for graph-based approximate nearest-neighbor search (ANNS). It includes a cycle-accurate simulator, dataset and index preparation utilities, CPU/GPU baseline implementations, fixed experiment configurations, and scripts for reproducing the key figures in the paper. The workflow reports retrieval quality, performance, and energy statistics, and generates the corresponding plots.

B. Artifact check-list (meta-information)

- **Program:** MFNNS simulator, HNSW CPU baseline, CAGRA GPU baseline, BANG, ANSMET, and the evaluated NMP design variants.
- **Compilation:** CMake, a C++20 compiler, CUDA, and Python.
- **Datasets:** Deep, GloVe, SIFT, Txt2Img, Word2Vec, Wiki, and PubMed datasets at the scales evaluated in the paper.
- **Run-time environment:** Linux, GCC, CUDA 12, Python 3.10, cuVS, CuPy, NumPy, and PyYAML.
- **Hardware:** An x86-64 server for simulation and CPU baselines; an NVIDIA GPU with at least 48 GiB memory for GPU baselines. Billion-scale experiments need a server with 2TB memory.
- **Disk space required:** Approximately 64 GiB for the standard workflow; billion-scale experiments require 2 TB.
- **Preparation time:** Approximately three days to download the datasets and build all indexes.
- **Experiment time:** Several hours with parallel execution. Individual large-scale simulation cases may take up to one day.
- **Publicly available:** Yes.
- **Workflow automation:** Shell scripts, CMake, Python, Conda, and optional Slurm submission.
- **Archived DOI:** <https://doi.org/10.5281/zenodo.21551985>

C. Description

1) *How to access:* The immutable artifact is archived at Zenodo: <https://doi.org/10.5281/zenodo.21551985>. We recommend you to access our GitHub repository for the latest version: <https://github.com/CLab-HKUST-GZ/micro59-mfnns>. After extracting the archive, all commands below should be executed from its root directory. The included `README.md` describes the directory layout and the expected output.

2) *Hardware and software dependencies:* The simulator and CPU experiments require Linux system, CMake, a C++20 compiler, Python 3, and at least 2 TB of memory. GPU experiments require CUDA 12 and an NVIDIA GPU with at least 48 GiB memory. The reference Python environment uses Python 3.10 with cuVS, CuPy, NumPy, and PyYAML. The simulator vendors its required C++ libraries in the artifact.

3) *Datasets:* The artifact provides download links, checksums, conversion utilities, and prepared-query specifications for all evaluated datasets. The standard workflow uses the seven non-billion-scale datasets. Billion-scale datasets are optional because of their storage and host-memory requirements.

D. Installation and preparation

The environment and input preparation workflow is:

- `dataset_prepare.sh`: Downloads and verifies the datasets, converts them to the required format, and prepares the queries and ground truth.
- `simulator_build.sh`: Builds the MFNNS cycle-accurate simulator and its dependencies.
- `cpu_index_build.sh` and `cagra/bang_index_build.sh`: Build the indexes required by the CPU and GPU baselines, respectively.

E. Experiment workflow

The artifact evaluation is organized into three main parts, covering hardware verification, index construction, and the reproduction of the paper results.

1. Functional verification of the MFNNS hardware:

- 1) **Create the Environment:** A pre-configured Docker image is provided with the required SpinalHDL, SBT, and RTL simulation dependencies.
- 2) **Run Verification:** Execute the complete functional test suite to verify the conventional FPMA, the square FPMA, and their bit-exact equivalence. The expected pass messages are listed in the corresponding README.

2. Index construction and Table V reproduction:

- 1) **Prepare Data and Indexes:** Use the scripts in `script/` to prepare the datasets and build the required CPU, BANG, and CAGRA indexes. Environment and storage requirements are documented in `mfnns_hnswlib/` and `GPU_Baseline/`.
- 2) **Run Evaluation:** Execute `run_figureXX.sh`. The data, detailed statistics, and logs are generated.

3. Simulator evaluation and figure reproduction:

- 1) **Validate or Run Experiments:** Follow the figure-specific instructions under `ae/figureXX/`. Archived results can be checked directly, while new simulator measurements can be executed locally or submitted as independent Slurm jobs.
- 2) **Generate Figures:** Run the command listed in README. The validated data, summaries, and final PDF/PNG figures are generated under `ae/figureXX/output/`.

F. Evaluation and expected results

A successful run creates a separate output directory for each figure, containing the exact commands, configuration files, raw logs, machine-readable statistics, and the reproduced plot. The evaluation scripts check that all formal runs process the expected queries and satisfy the matched-recall target used in the paper. Absolute GPU timing may vary across machines, but the recall targets, relative performance ordering, and principal efficiency trends should agree with the reported results.

REFERENCES

- [1] A. Barman and S. K. Shah, "A graph-based approach for making consensus-based decisions in image search and person re-identification," *IEEE transactions on pattern analysis and machine intelligence*, vol. 43, no. 3, pp. 753–765, 2019.
- [2] J. Park, B. Kim, S. Yun, E. Lee, M. Rhu, and J. H. Ahn, "Trim: Enhancing processor-memory interfaces with scalable tensor reduction in memory," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 268–281.
- [3] S. Li, Y. Wang, E. Hanson, A. Chang, Y. S. Ki, H. Li, and Y. Chen, "Ndrec: A near-data processing system for training large-scale recommendation models," *IEEE Transactions on Computers*, vol. 73, no. 5, pp. 1248–1261, 2024.
- [4] Y. Meng, X. Dai, X. Yan, J. Cheng, W. Liu, J. Guo, B. Liao, and G. Chen, "Pmd: An optimal transportation-based user distance for recommender systems," in *European Conference on Information Retrieval*. Springer, 2020, pp. 272–280.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.
- [6] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millikan, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark *et al.*, "Improving language models by retrieving from trillions of tokens," in *International conference on machine learning*. PMLR, 2022, pp. 2206–2240.
- [7] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 9802–9822.
- [8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.
- [9] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, and Y. Wang, "Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 4236–4247.
- [10] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A scalable processing-in-memory accelerator for parallel graph processing," in *Proceedings of the 42nd annual international symposium on computer architecture*, 2015, pp. 105–117.
- [11] M. Gao, G. Ayers, and C. Kozyrakis, "Practical near-data processing for in-memory analytics frameworks," in *2015 International Conference on Parallel Architecture and Compilation (PACT)*. IEEE, 2015, pp. 113–124.
- [12] A. Boroumand, S. Ghose, Y. Kim, R. Ausavarungrun, E. Shiu, R. Thakur, D. Kim, A. Kuusela, A. Knies, P. Ranganathan *et al.*, "Google workloads for consumer devices: Mitigating data movement bottlenecks," in *Proceedings of the twenty-third international conference on architectural support for programming languages and operating systems*, 2018, pp. 316–331.
- [13] G. Dai, T. Huang, Y. Chi, J. Zhao, G. Sun, Y. Liu, Y. Wang, Y. Xie, and H. Yang, "Graphh: A processing-in-memory architecture for large-scale graph processing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 4, pp. 640–653, 2018.
- [14] Y. Zhuo, C. Wang, M. Zhang, R. Wang, D. Niu, Y. Wang, and X. Qian, "Graphq: Scalable pim-based graph processing," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 712–725.
- [15] L. Wu, H. Zhu, S. He, X. Lin, X. Zeng, and C. Chen, "Pimoe: Towards efficient moe transformer deployment on npu-pim system through throttle-aware task offloading," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2025, pp. 1–7.
- [16] Y. Li, Y. Jin, B. Tian, H. Zhang, and M. Gao, "Ansmet: Approximate nearest neighbor search with near-memory processing and hybrid early termination," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1093–1107.
- [17] C. Liu, H. Liu, D. Chen, Y. Huang, Y. Zhang, W. Xiao, X. Liao, and H. Jin, "Heterrag: Heterogeneous processing-in-memory acceleration for retrieval-augmented generation," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 884–898.
- [18] Z. Zhang, C. Jin, L. Tang, X. Liu, and X. Jin, "Fast, approximate vector queries on very large unstructured datasets," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 995–1011.
- [19] J. Gao and C. Long, "High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–27, 2023.
- [20] M. Chatzakis, Y. Papakonstantinou, and T. Palpanas, "Darth: Declarative recall through early termination for approximate nearest neighbor search," *Proceedings of the ACM on Management of Data*, vol. 3, no. 4, pp. 1–26, 2025.
- [21] Y. Chen, J. Zou, and X. Chen, "April: Accuracy-improved floating-point approximation for neural network accelerators," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2025, pp. 1–7.
- [22] J. Zou, Y. Chen, X. Chen, C. Xu, and X. Chen, "Axcure: A quantization-aware approximate gemm unit for llm inference," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 839–853.
- [23] S. Morozov and A. Babenko, "Non-metric similarity graphs for maximum inner product search," *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [24] V. Karthik, S. Khan, S. Singh, H. V. Simhadri, and J. Vedurada, "Bang: Billion-scale approximate nearest neighbour search using a single gpu," *IEEE Transactions on Big Data*, 2025.
- [25] Y. Fu, C. Chen, Y. Chen, W.-F. Wong, and B. He, "Geometric partition for billion-scale approximate nearest neighbor search," *IEEE Transactions on Knowledge and Data Engineering*, 2026.
- [26] K. L. Clarkson, "An algorithm for approximate closest-point queries," in *Proceedings of the tenth annual symposium on Computational geometry*, 1994, pp. 160–164.
- [27] H. V. Simhadri, G. Williams, M. Aumüller, M. Douze, A. Babenko, D. Baranchuk, Q. Chen, L. Hosseini, R. Krishnaswamy, G. Srinivasa *et al.*, "Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search," in *NeurIPS 2021 Competitions and Demonstrations Track*. PMLR, 2022, pp. 177–189.
- [28] H. Jégou, R. Tavenard, M. Douze, and L. Amsaleg, "Searching in one billion vectors: re-rank with source coding," in *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2011, pp. 861–864.
- [29] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin, "Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement," *IEEE Transactions on Knowledge and Data Engineering*, vol. 32, no. 8, pp. 1475–1488, 2019.
- [30] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *IEEE Transactions on Big Data*, 2025.
- [31] W. Dong, C. Moses, and K. Li, "Efficient k-nearest neighbor graph construction for generic similarity measures," in *Proceedings of the 20th international conference on World wide web*, 2011, pp. 577–586.
- [32] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," *Proceedings of the VLDB Endowment*, vol. 12, no. 5, pp. 461–474, 2019.
- [33] M. Wang, H. Wu, X. Ke, Y. Gao, Y. Zhu, and W. Zhou, "Accelerating graph indexing for anns on modern cpus," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–29, 2025.
- [34] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 1964–1978, 2021.
- [35] J. Jang, H. Choi, H. Bae, S. Lee, M. Kwon, and M. Jung, "CXL-ANNS: Software-Hardware collaborative memory disaggregation and computation for Billion-Scale approximate nearest neighbor search," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 585–600.
- [36] D. Quinn, M. Nouri, N. Patel, J. Salihu, A. Salemi, S. Lee, H. Zamani, and M. Alian, "Accelerating retrieval-augmented generation," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 15–32.
- [37] D. Quinn, E. E. Yücel, M. Prammer, Z. Fan, K. Skadron, J. M. Patel, J. F. Martínez, and M. Alian, "Drex: Accurate and scalable dense retrieval acceleration via algorithmic-hardware codesign," in *Proceedings of the*

- 52nd Annual International Symposium on Computer Architecture, 2025, pp. 1108–1124.
- [38] P. Chen, W.-C. Chang, J.-Y. Jiang, H.-F. Yu, I. Dhillon, and C.-J. Hsieh, “Finger: Fast inference for graph-based approximate nearest neighbor search,” in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 3225–3235.
- [39] C. Li, M. Zhang, D. G. Andersen, and Y. He, “Improving approximate nearest neighbor search through learned adaptive early termination,” in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 2539–2554.
- [40] Synopsys, Inc., “DesignWare Intellectual Property,” 2026, [Online]. Available: <https://www.synopsys.com/dw/buildingblock.php> [Accessed: Feb. 2026].
- [41] J. N. Mitchell, “Computer multiplication and division using binary logarithms,” *IRE Transactions on Electronic Computers*, no. 4, pp. 512–517, 2009.
- [42] O. Gustafsson and N. Hellman, “Approximate floating-point operations with integer units by processing in the logarithmic domain,” in *2021 IEEE 28th Symposium on Computer Arithmetic (ARITH)*. IEEE, 2021, pp. 45–52.
- [43] H. Wang, J. Fang, X. Tang, Z. Yue, J. Li, Y. Qin, S. Guan, Q. Yang, Y. Wang, C. Li *et al.*, “Sofa: A compute-memory optimized sparsity accelerator via cross-stage coordinated tiling,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 1247–1263.
- [44] W. Sun, Z. Li, S. Yin, S. Wei, and L. Liu, “Abc-dimm: Alleviating the bottleneck of communication in dimm-based near-memory processing with inter-dimm broadcast,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 237–250.
- [45] Z. Zhou, C. Li, F. Yang, and G. Sun, “Dimm-link: Enabling efficient inter-dimm communication for near-memory processing,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 302–316.
- [46] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, “Ramulator 2.0: A modern, modular, and extensible dram simulator,” *IEEE Computer Architecture Letters*, vol. 23, no. 1, pp. 112–116, 2023.
- [47] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, “Cacti 7: New tools for interconnect exploration in innovative off-chip memories,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.
- [48] G. Dai, Z. Zhu, T. Fu, C. Wei, B. Wang, X. Li, Y. Xie, H. Yang, and Y. Wang, “Dimmining: pruning-efficient and parallel graph mining on near-memory-computing,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 130–145.
- [49] R. Mahapatra, H. Santhanam, C. Priebe, H. Xu, and H. Esmaeilzadeh, “In-storage acceleration of retrieval augmented generation as a service,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 450–466.
- [50] Z. Liu, W. Ni, J. Leng, Y. Feng, C. Guo, Q. Chen, C. Li, M. Guo, Y. Ma, F. Zhang *et al.*, “Juno++: Optimizing anns and enabling efficient sparse attention in llm via ray tracing core,” *ACM Transactions on Architecture and Code Optimization*, vol. 22, no. 4, pp. 1–25, 2025.
- [51] B. Zheng, Z. Yue, Q. Hu, X. Yi, X. Luan, C. Xie, X. Zhou, and C. S. Jensen, “Learned probing cardinality estimation for high-dimensional approximate nn search,” in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 3209–3221.
- [52] S. Zeng, Z. Zhu, J. Liu, H. Zhang, G. Dai, Z. Zhou, S. Li, X. Ning, Y. Xie, H. Yang *et al.*, “Df-gas: a distributed fpga-as-a-service architecture towards billion-scale graph-based approximate nearest neighbor search,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 283–296.
- [53] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, “Diskann: Fast accurate billion-point nearest neighbor search on a single node,” *Advances in neural information processing Systems*, vol. 32, 2019.
- [54] H. Guo and Y. Lu, “Achieving low-latency graph-based vector search via aligning best-first search algorithm with ssd,” *ACM Transactions on Storage*, vol. 22, no. 2, pp. 1–30, 2026.
- [55] S. Liang, Y. Wang, Z. Yuan, C. Liu, H. Li, and X. Li, “Vstore: In-storage graph based vector search accelerator,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 997–1002.
- [56] Y. Wang, S. Li, Q. Zheng, L. Song, Z. Li, A. Chang, H. Li, and Y. Chen, “Ndsearch: Accelerating graph-traversal-based approximate nearest neighbor search through near data processing,” in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture*, 2024, pp. 368–381.
- [57] B. Tian, H. Liu, Z. Duan, X. Liao, H. Jin, and Y. Zhang, “Scalable billion-point approximate nearest neighbor search using smartssds,” in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 2024, pp. 1135–1150.
- [58] K. Chen, R. Nadig, A. K. Kakolyris, M. Frouzakakis, N. Mansouri Ghiasi, Y. Liang, H. Mao, J. Park, M. Sadrosadati, and O. Mutlu, “Reis: A high-performance and energy-efficient retrieval system with in-storage processing,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1171–1192.
- [59] H.-W. Hu, W.-C. Wang, Y.-H. Chang, Y.-C. Lee, B.-R. Lin, H.-M. Wang, Y.-P. Lin, Y.-M. Huang, C.-Y. Lee, T.-H. Su *et al.*, “Ice: An intelligent cognition engine with 3d nand-based in-memory computing for vector similarity search acceleration,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 763–783.
- [60] Heterogeneous Accelerated Compute Cluster (HACC) at NUS. [Online]. Available: <https://xacchead.ddns.comp.nus.edu.sg/>. Accessed: Sep 2026.